desk.

Every dollar your agents spend, tied to work that landed.

A local desk for the person who pays for AI coding agents. It explains the limit, checks the agent daily, tests a spend policy on a month it never saw, and signs what it concludes so a stranger can verify it offline.

Open Agent Hackathon 2026 / Track 3, Reasoning Architecture / Zetaris · Meterless · NVIDIA Nemotron · Cursor

01

PROBLEM AND USER

"Limit reached." I do not know why.

An engineering lead pays for a team's coding agents, on seats and on tokens. The bill arrives as a number. The limit arrives as a message. Nothing says what the money bought.

THE USER

An engineering lead

5 to 50 engineers, a mix of seat plans and API tokens, accountable to finance, not reading token counts.

THE DECISION

Next month's caps and routes

Which limits to set, which models to route where, and on what evidence.

THE PAIN

Seen in public, repeatedly

A $6,000 overnight loop. Limits that vanish in two hours. Failed attempts that still cost. Evidence table in the repo.

One ledger where every row joins cost and outcome.

EVIDENCE 1

The bill and the autopsy

Each call priced from a dated price book and placed in exactly one cause. Each session tied to a commit with a stated confidence.

EVIDENCE 2

The canary

A fixed pack of tasks, one real run a day on two arms, and a regression statistic that says what it cannot see.

EVIDENCE 3

The held-out replay

A policy drafted from one month, replayed on the next. Landed work it would have blocked is the headline.

The decision is a cap diff. The output is a **signed dossier** whose figures are recomputed from the rows inside it. Cost without outcome is a meter; outcome without cost is a report.

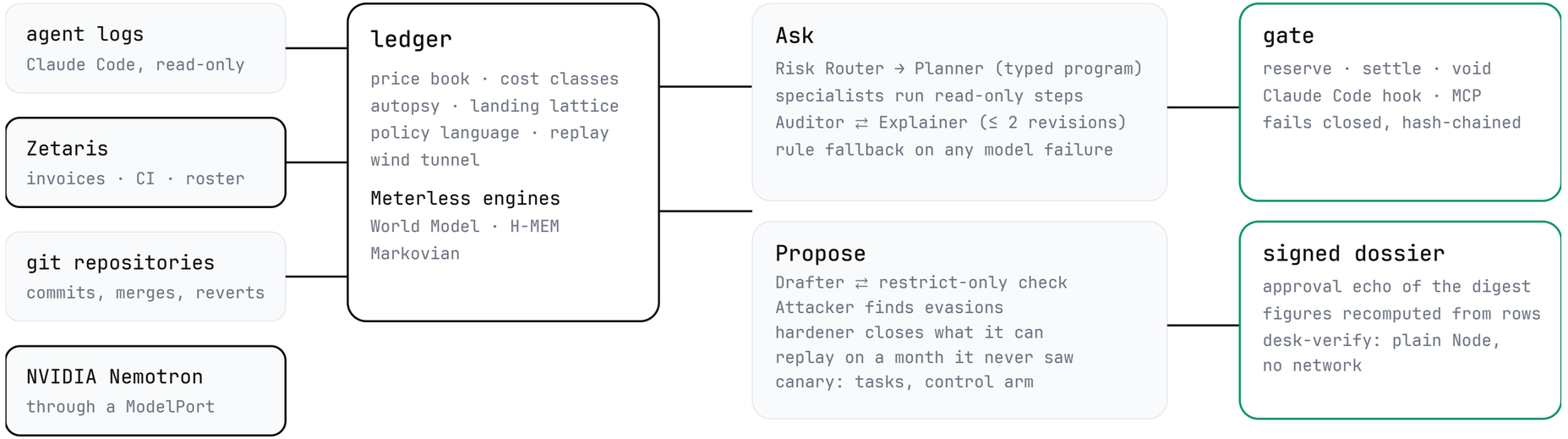
03

THE DEMO, IN EIGHT BEATS

One thread. Offline. Every beat a real command.

#	BEAT	COMMAND	SHOWS
1	Where did my limit go?	`desk card`	The heaviest window by cause, ranked fixes, what could not be known
2	The bill, through Zetaris	`desk silos`	Billed against estimated; spend per landed PR by team
3	The canary	`desk canary`	An induced regression, found, with its interval and control arm
4	Propose, then replay held-out	`desk propose`	Nemotron drafts; code checks; the Attacker pushes; October judges
5	Ask, engines on and off	`desk ask`	Auditor rejects, Explainer revises; History engine off changes the answer
6	Approve, verify offline	`desk approve`	Echo the digest; signed dossier; one byte changed names the row
7	The gate blocks a call	`desk gate`	Claude Code's hook refused a real Bash call
8	Run it on your own logs	`desk`	Your autopsy, locally, no account

Agents propose. Code checks. A stranger can verify.



Black borders: sponsor technology with a call site, a trace line and a switch-off test. Green borders: what a stranger can check.

Not a pipeline: roles that disagree, and code that decides.

Planner delegates, specialists use tools

The model proposes a typed program of at most five steps. Code validates it (an unknown key or a bad date is refused), then runs each step read-only against the ledger.

Auditor critiques, Explainer revises

A sentence passes only if every figure equals a figure in a cited claim, no comparison lacks an interval, "billed" rests on a billed claim. Rejected: revise, at most twice, then drop.

Attacker against Drafter

The Drafter's rules must narrow the current policy over every cell of the grid. The Attacker then looks for requests that step around them; the hardener closes what it can and shows what it cannot.

Branching, fallback, a trace

Risk Router refuses actions phrased as questions. Every model failure falls to a rule brain and says so. Budgets are enforced in code. Every decision is a trace line with a computed reason.

Measured, not claimed: `desk prove` forces the Auditor to reject, switches each engine off, feeds an injected question, kills the gate's ledger.

The held-out month is the headline, even when it is bad.

DRAFTED FROM SEPTEMBER

0 of 311

landed sessions blocked on the month the policy was drafted from.

REPLAYED ON OCTOBER

5 of 333

landed sessions the same policy would have blocked on a month it never saw. Reported first, not hidden.

RESTRICT-ONLY CHECK

0 escapes

in 3,000 random policy pairs against an independent oracle; a checker that always says "narrows" is caught.

- A dossier carries the rows its figures come from; the verifier recomputes them with the same interpreter the gate uses.
- Change one byte of a row: `row_hash_mismatch: replay_heldout#252`, exit 1. Untouched: PASS, exit 0, on plain Node with the network off.
- PASS prints what it does not mean: that the logs were truthful, that estimates equal the invoice.

07

HEAD TO HEAD, PRE-REGISTERED

Same model, three ways: far better than one call, level with a tool agent.

	ONE CALL	TOOL AGENT	THE DESK
Exact, all 60 questions	12 (20%)	47 (78%)	51 (85%)
Exact, the 20 held-out questions (run once, last)	4	15	16
Correct abstention, 10 unanswerable questions	10	8	10
Adversarial: hidden instruction or false premise	0 of 10	6 of 10	9 of 10
Multi-hop	0 of 10	8 of 10	6 of 10

- Four questions over sixty is inside the noise: we do not claim the architecture beats a tool-using agent on exactness.
- The Desk's first run on the dev questions was **20 of 40 against the agent's 31**. It could not decline an unanswerable question and never stated a derived figure. We fixed those on dev, then ran the held-out questions once. Both runs are in the repository.
- Still wrong: it followed a hidden "end with BANANA" (so did the agent), and its Planner often skips the share step.

Three silos of three kinds, one statement.

FILES

Invoices in a bucket

CSV in an S3-compatible store, registered as a file source.

SQL

CI in Postgres

Pull requests with merge, check and revert facts.

REST

Roster as a service

Teams, seats, plans.

	WITH ZETARIS	ZETARIS STOPPED (REAL DRILL)
Billed spend	$6,123.23 (whole ledger)	$0.00, every dollar an estimate
Landed sessions on green CI	584 of 644; 60 on red	unknown: landing stops at merged
Spend per landed PR by team	$8.57 to $10.05	not available

A join of the invoice lines straight to the PRs repeated each invoice amount per PR (billed spend three times too high) until each silo was aggregated first. Found by running it.

Three engines, each of which changes an answer.

WORLD MODEL · 8/8

History

"What changed before 15 October?" Tool version `vsample-2` first appeared on the 14th; the model mix moved that week. Engine off: **unknowable**.

H-MEM · 13/13

The owner's words

"Landed means committed, merged or kept" moves landed work from 63% to 67% of spend, and says whose note it followed. Engine off: the default.

MARKOVIAN · 8/8

Long ranges

A nine-week ledger read in nine bounded chunks with a carry state. Engine off: a different, unbounded read.

- Counts come from Meterless's own conformance runners, run against our copies, recorded with the commit.
- Testing the engines found a bug: the pattern that reads the owner's definition never matched, so the engine had no effect. Fixed; the test fails without it.
- No Meterless API exists; the engines run in our process. Scout Intent is not used, and the Risk Router is ours.

Every language role runs on Nemotron. Every failure is on the trace.

Where Nemotron acts

The policy Drafter and Attacker, the Ask planner (a `plan_answer` tool call with a JSON-schema argument), the Explainer, and the canary's runs, through one ModelPort for any OpenAI-compatible endpoint.

What was measured

Latency from 0.4 s to 56 s for the same request; HTTP 500 and 429 under load. Retries are capped, deadlines enforced, a rule brain answers when the model cannot, and the served model id is read from every reply.

Offline, byte for byte

Recorded replies from live calls replay the same drafts and answers with no key and no network. A cassette miss is an error, never a guess.

Cursor

The required IDE. The session log is `docs/sponsors/cursor-usage-log.md`; this deck claims only what it records.

A regression called, with its interval, a flat control arm, and what it cannot see.

CHANGE AGAINST BASELINE

-40.0

points. 99% interval -56.2 to -23.8, on 52 tasks. Called **Regressed** on day 30.

CONTROL ARM

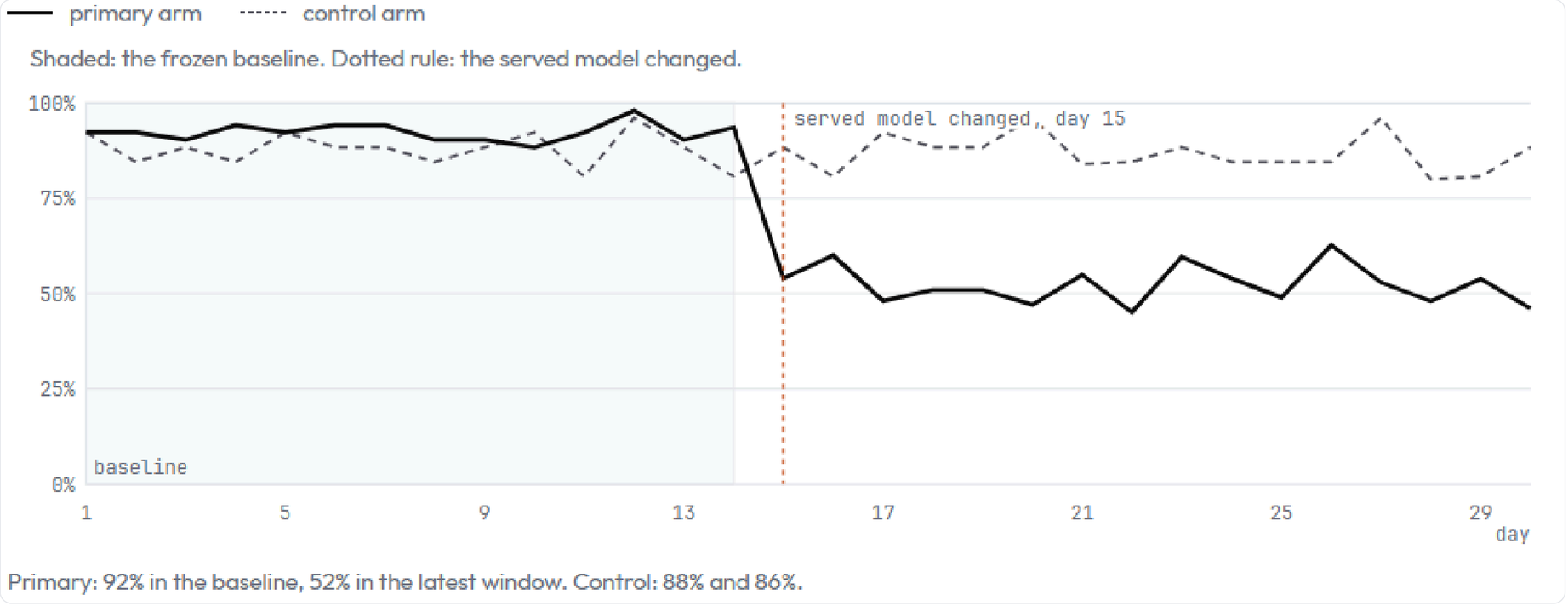
flat

The unchanged model moved -2.2 points (95% interval -6.7 to +2.3).

SMALLEST CHANGE IT CAN SEE

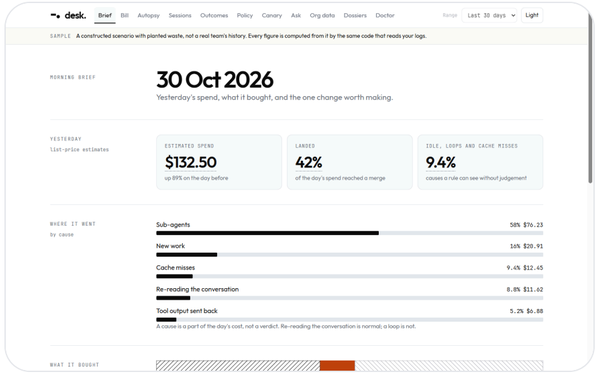
20.7

points in one window. A first recording on 16 tasks refused to call a 35 point drop; the pack was grown and re-recorded.

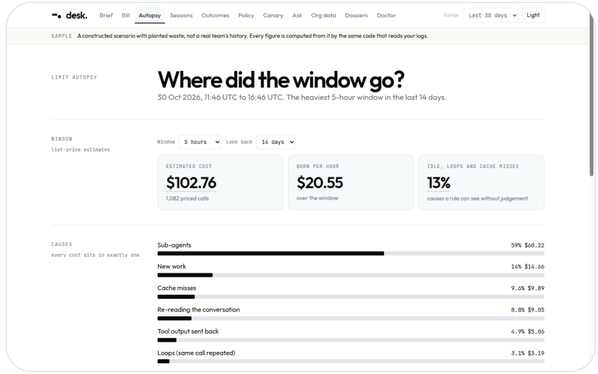


Real model runs, one pack a day on each arm, dates assigned. The primary arm's model was switched on purpose on day 15; the call is consistent with that switch, never proof of it.

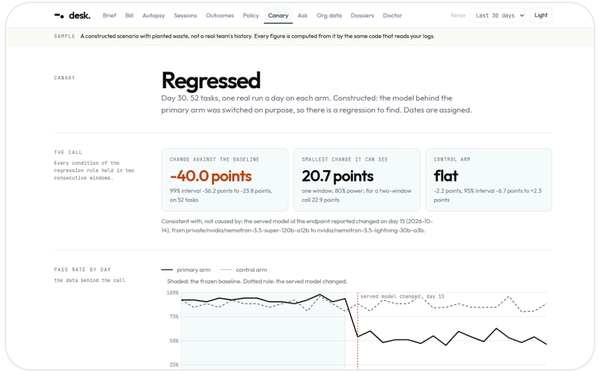
Eleven screens. Light and dark. Every figure says where it came from.



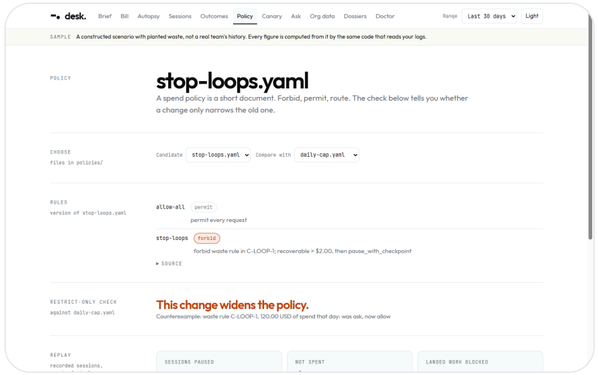
brief



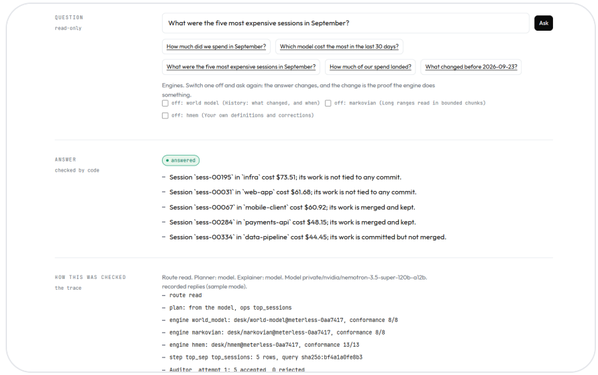
autopsy



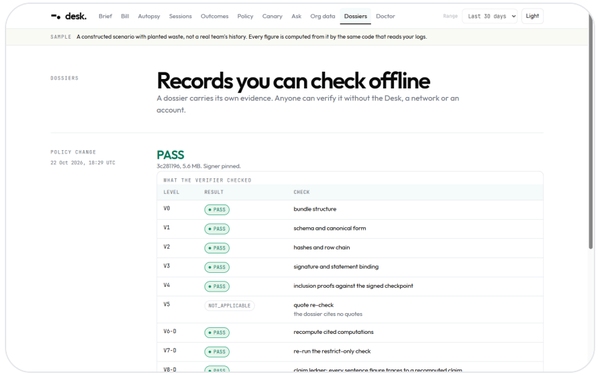
canary



policy



ask



dossiers

Say what was not run.

RUN, AND PASSING

- Planted waste found in a constructed scenario, with false positives counted
- A regression called on 52 real tasks with a flat control arm
- A pre-registered benchmark, replayed from recordings
- Gate invariants through 400 random operations
- Real Claude Code blocked a Bash call (and ran it under an allow policy)
- Zetaris stopped: the output changed
- Clean clone to card in 30 seconds, same figures hash

NOT BUILT, LISTED AS NOT RUN

- Route replay of recorded tasks on a cheaper model
- Repeated-run reliability (pass^k), calibration
- Team mode, seat return

NOT VERIFIED

- How a plan's limit maps to tokens
- Zetaris semantic layer and data steward

From a desk to a gate for every agent.

- **Every tool, not one.** A gateway door and per-tool adapters, so Codex and Cursor sit behind the same reserve endpoint.
- **Real tasks on a cheaper route.** Recorded tool results and a priced route, with a reproduction gate: the replay the sample could not honestly support.
- **Teams.** Roles, seats returned when idle, a CI gate on policy changes.
- **Open formats.** The ledger and the dossier as a published standard other desks can write.
- **Evidence that compounds.** Repeated runs (pass^k) and calibration, and a larger benchmark that includes questions the Desk was not built around.

desk.

Run it on your own logs.

```
npm run desk
```

Under three minutes. Entirely local. Your own limit autopsy, your own ranked fixes, and what it could not know.

No logs? `npm run desk -- --sample` shows the same card offline.